

TP1 - Recorte consciente de audio

Motivación

Acortar una grabación para que entre en una duración determinada es una tarea cotidiana en la producción musical y audiovisual. El enfoque más sencillo, truncar, corta el final de la pieza y descarta justamente el cierre. Acelerar la reproducción de manera uniforme, por su parte, altera el tempo y la altura de todo el material, y por encima de cierto factor el resultado deja de ser aceptable para el oído. Frente a estas limitaciones, el recorte *consciente del contenido* [1] propone algo distinto: en lugar de comprimir la pieza entera, se descartan fragmentos y se unen los extremos aprovechando que la música se repite. Si el salto cae en un punto donde el material que sigue es muy parecido al que debería haber sonado, el corte resulta inaudible. Esta técnica se aplica desde hace tiempo en la producción de los *radio edits*, las versiones acortadas que se emiten por radio, y hoy aparece incorporada en editores de audio y de video. La idea central es preservar la estructura de la pieza, sus estrofas, estribillos y cierres, en lugar de recortarla por un extremo.

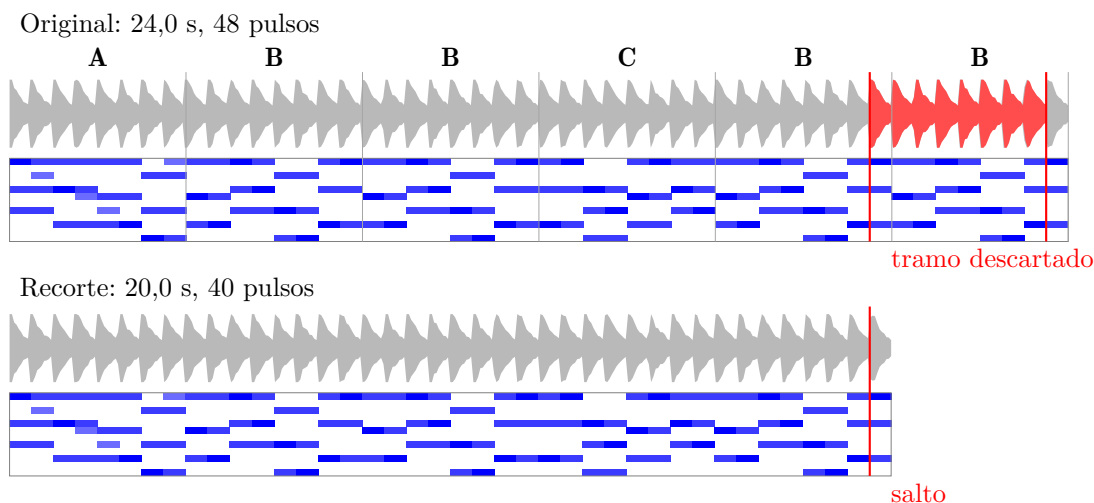


Figura 1: Ejemplo de recorte consciente sobre una grabación con estructura A B B C B B: forma de onda original con el tramo descartado marcado en rojo (arriba), y resultado tras unir los pulsos conservados (abajo). Debajo de cada forma de onda se grafican los vectores de características, un vector por pulso. La envolvente de la señal no permite distinguir una sección de otra, pero las características sí, y muestran que las cuatro repeticiones de B son idénticas entre sí y distintas de A y de C. El algoritmo descarta un estribillo completo, de modo que el único salto cae entre dos repeticiones idénticas y no se percibe.

El problema del recorte consciente

Una grabación se representa como una secuencia de n **pulsos**, donde cada pulso i tiene asociado un vector de características $f_i \in \mathbb{R}^d$ que resume su contenido armónico y tímbrico en ese instante. Una **selección** de k pulsos es una secuencia de índices

$$1 = i_1 < i_2 < \dots < i_k = n,$$

es decir, el primer y el último pulso siempre se conservan, de manera que el recorte empieza y termina donde lo hace la grabación original. Reproducir el pulso i y saltar a continuación al pulso j tiene un costo

$$c(i, j) = \|f_{i+1} - f_j\|_2,$$

que mide cuán distinto es el pulso al que efectivamente se salta respecto del que debería haber sonado a continuación. En particular $c(i, j) = 0$ cuando $j = i + 1$, es decir, cuando no hay corte alguno. El **costo de la selección** se define como

$$C = \sum_{t=1}^{k-1} c(i_t, i_{t+1}).$$

El objetivo del problema es encontrar la selección de k pulsos de mínimo costo.

Los datos

El formato de los archivos de entrada es el siguiente:

1. **Primera línea:** Tres enteros n , d y k , indicando la cantidad de pulsos de la grabación, la cantidad de características de cada pulso, y la cantidad de pulsos a conservar, respectivamente.
2. **Las siguientes n líneas:** Cada línea contiene d valores reales $f_{i,1}, f_{i,2}, \dots, f_{i,d}$, representando las características del pulso i .

A modo de ejemplo, el siguiente archivo representa una grabación de 6 pulsos con una única característica por pulso, de la que hay que conservar 4:

```
6 1 4
0.0
1.0
5.0
6.0
2.0
3.0
```

- $n = 6$, $d = 1$, $k = 4$: la grabación tiene 6 pulsos y hay que conservar 4 de ellos.
- Cada pulso se describe con un único valor real. Por ejemplo, la característica del pulso 3 es 5,0.
- La selección de mínimo costo para esta instancia es 1, 2, 4, 6 (indexado desde 1), con costo total $0,0 + 1,0 + 1,0 = 2,0$. Las selecciones 1, 2, 3, 6 y 1, 2, 5, 6 tienen ambas costo 3,0.

La salida esperada es una línea con k enteros $i_1, i_2, \dots, i_k$ (indexados desde 1), indicando los pulsos conservados en orden creciente.

En la práctica los vectores de características son cromas de $d = 12$ dimensiones, extraídos de una grabación real, y n está en el orden de los cientos o miles de pulsos.

La entrega incluye además dos scripts auxiliares, `extraer.py` y `reconstruir.py`, que convierten una grabación en una instancia del problema y reconstruyen el audio a partir de una selección de pulsos. No forman parte de lo que hay que implementar ni de lo que se evalúa: están para que el grupo pueda recortar sus propias grabaciones y escuchar cómo suena el resultado.

El trabajo

El trabajo práctico debe ser realizado en grupos de exactamente **3** personas. Se pide:

1. **Algoritmos exactos.** Implementar los siguientes algoritmos en C++:
 - **Fuerza bruta (FB).** Implementar una solución para el problema que utilice la técnica de fuerza bruta. Describir en el informe el pseudocódigo y analizar la complejidad del algoritmo.
 - **Backtracking (BT).** Diseñar un algoritmo de backtracking con al menos una poda por factibilidad y una poda por optimalidad. Describir en el informe el pseudocódigo, las podas propuestas y la complejidad.
 - **Programación dinámica (PD).** Utilizar las soluciones construidas previamente para diseñar un algoritmo de programación dinámica, incluyendo la reconstrucción de la selección óptima. Analizar la complejidad temporal y espacial.
2. **Implementación en Python.** Re-implementar los algoritmos de BT y PD en Python. En este punto, es posible utilizar asistencia de IA (ChatGPT, Claude, Gemini, etc.). Incluir en el informe una breve descripción de la herramienta utilizada, cómo fue empleada y el enlace correspondiente a la conversación.
3. **Experimentación y discusión.** Realizar una experimentación exhaustiva usando datasets propios, justificando la elección de los mismos.

En base a los resultados, establecer comparaciones entre los algoritmos y entre las implementaciones para responder: ¿cuál es la recomendación del grupo respecto al algoritmo, la implementación y el lenguaje de programación a utilizar en la práctica?
4. **Diseño e implementación.** La entrega incluye un conjunto de clases base que representan las entidades principales del problema (`Instancia`, `Algoritmo` y `Solucion`), que el grupo puede extender y modificar según considere conveniente. El diseño final debe estar justificado. Sin embargo, la *signature* de los métodos de las clases base no debe ser modificada.
5. **Informe, presentación de resultados y entrega del código.** La descripción de los algoritmos, las decisiones de diseño, el testing realizado, la presentación de resultados, e instrucciones de compilación y ejecución.

Modalidad de entrega

Se pide presentar el modelo y la experimentación en un informe de máximo 15 páginas que contenga:

- introducción al problema,
- descripción de los algoritmos propuestos e implementados,
- consideraciones generales respecto a la implementación, incluyendo dificultades encontradas,
- resumen de resultados obtenidos en la experimentación,
- conclusiones, posibles mejoras y observaciones adicionales que consideren pertinentes.

Junto con el informe debe entregarse el código con la implementación. El mismo debe ser entendible, incluyendo comentarios que faciliten su corrección y ejecución.

Fechas de entrega

Formato Electrónico: **18 de septiembre de 2026, 23:59 hs**, enviando el trabajo (informe + código) vía el campus virtual.

Importante: El horario es estricto. Las entregas recibidas después de la hora indicada serán consideradas re-entrega.

Referencias

- [1] Simon Wenner, Jean-Charles Bazin, Alexander Sorkine-Hornung, Changil Kim, and Markus Gross. Scalable music: Automatic music retargeting and synthesis. *Computer Graphics Forum*, 32(2):345–354, 2013.